

Softwares de Sistemas

Conceitos sobre Linguagem Java

SOFTWARE BÁSICO, TURMA A
PROF. MARCELO LADEIRA CIG/UNB

BASEADO NO LIVRO "THE JAVATM VIRTUAL MACHINE SPECIFICATION", SECOND EDITION,
TIM LINDHOLM & FRANK YELLIN.

Introdução

Linguagem Java

- Sintaxe similar à da C
 - “Fortemente tipada”
 - Tipo booleano
- Gerência de memória heap
 - Coletor de lixo.
 - Facilidade de uso
 - Não permite referência pendente
 - Uso de referências
 - Referências tipadas
 - Tempo não determinístico
 - Não indicado para tempo real

Linguagem C

- Linguagem “fortemente tipada”
 - Sem tipos lógicos.
- Gerência de heap
 - A cargo do programador
 - Maior eficiência
 - Permite referência pendente
 - Uso de ponteiros
 - Ponteiros tipados
 - Tempo determinístico
 - Indicado para tempo real

Introdução

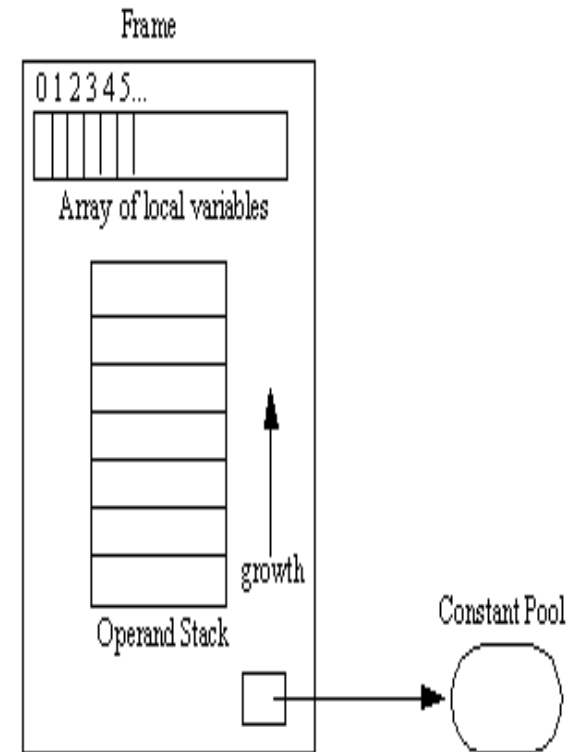
Máquina Virtual Java

- Responsável por prover
 - Independência do hardware e sistema operacional
 - Código compilado compacto
 - Proteção ao cliente contra programas maliciosos
- Máquina abstrata
 - entrada: bytecode no formato .class
 - código de saída
 - Interpretado ou compilado para código da máquina hospedeira
 - Pode ser implementado em micro-código ou silício.
 - manipula várias áreas de memória em tempo de execução
 - Não possui registradores
 - Máquina de pilha

Introdução

Máquina Virtual Java

- Cada thread tem uma pilha JVM que armazena *frames*
 - um frame é criado quando um método é invocado
 - uma pilha de operandos de bytecodes
 - um vetor de variáveis locais do método
 - um ponteiro (referência) para o pool de constante em *runtime* da classe do método corrente



Introdução

Máquina Virtual Java

▪ Vetor (tabela) de variáveis locais

- contêm os parâmetros dos métodos e valores das variáveis locais
 - Os **parâmetros são armazenados primeiro**, iniciando no índice 0.
 - se o frame é para **um método construtor ou de instância**, o ponteiro é armazenado no índice 0 e é seguido pelos parâmetros.
 - se é para **um método estático**, o primeiro parâmetro formal é armazenado no índice 0, e assim por diante.
- tamanho determinado em tempo de compilação
 - função do número e tamanho das variáveis locais e parâmetros formais

▪ Pilha de operandos (32 bits)

- usada para empilhar e desempilhar valores operandos de *opcode*.
- tamanho determinado em tempo de compilação.
 - algumas instruções empilham valores na pilha.
 - outras retiram operandos da pilha, manipula-os e empilham o resultado.
 - valores retornados pelos métodos são recebidos via pilha de operandos.

Introdução

Máquina Virtual Java

■ Pool de constantes

- armazena todas as constantes associadas com a classe
 - inclui os valores das variáveis declaradas como “final”
- Contribui para tornar menores os requisitos de memória pois evita a duplicação de constantes em cada instância (objetos) de uma classe.
 - opcode que empilham constantes a partir do pool utilizam um “índice de pool de constante”
 - Operando de 1 byte (1 a 255) ou 2 bytes (1 a 65535) que segue o código do código de operação (*opcode*)

Tipos e Valores: Tipos Primitivos

Tipo	Definição
byte	1-byte com sinal convertido em inteiro de 32 bits
short	2-bytes com sinal convertidos em inteiro de 32 bits
int	4-bytes com sinal (inteiro de 32 bits)
long	8-bytes com sinal convertido em 2 inteiros de 32 bits
float	4-bytes IEEE 754 ponto flutuante de precisão simples
double	8-bytes IEEE 754 ponto flutuante de precisão dupla
char	2-bytes sem sinal representando um caracter Unicode

Tipos e Valores

Tipos de Ponto Flutuante, Conjuntos de Valores e Valores

- Valores especiais definidos nas classes Float e Double

Float.NaN

Float.NEGATIVE_INFINITY

Float.POSITIVE_INFINITY

Double.NaN

Double.NEGATIVE_INFINITY

Double.POSITIVE_INFINITY

Tipos de Variáveis

Variável de classe (globais às instâncias)

- É field de um tipo de classe declarado como *static* em declaração de classe ou com ou sem *static* em declaração de interface.
 - criada quando a classe ou interface é carregada e iniciada com valores *default* e desalocada quando a classe ou interface é removida.

Variável de instância (locais às instâncias)

- É field declarado em uma declaração de classe sem uso de *static*.
 - criada quando um objeto ou uma subclasse dessa classe é criada e iniciada com valores *default*.
 - desalocada se não mais existir referência ao objeto que a contém.

Componentes de array

- São variáveis anônimas criadas e iniciadas com valores *default* quando um novo objeto do tipo array é criado.
 - componentes do array deixam de existir se o array não for mais referenciado.

Parâmetros de métodos (locais aos métodos)

- Nomes de valores de argumentos passados para um método.

Parâmetros de construtores (locais aos construtores)

Nomes de valores de argumentos passados para método construtor.

Tipos de Variáveis

Parâmetro de manipulador de exceção (local ao manipulador)

- criado cada vez que uma exceção é capturada por uma cláusula *catch* de um comando *try*.
 - a nova variável é iniciada com o objeto atual associado com a exceção.
 - o parâmetro deixa de existir quando a execução do bloco associado com a cláusula *catch* é completada.

Variáveis locais (locais a blocos ou comando *for*)

- São declaradas por comandos de declaração de variáveis locais em blocos ou em um comando *for*.
 - uma nova variável é criada para cada variável local declarada.
 - não é iniciada até que o comando que a declara seja executado.
 - a variável local deixa de existir quando a execução do bloco ou comando *for* é completada.

Nomes e Pacotes

- **Nomes são usados para referir as entidades declaradas no programa**
 - pacote, tipo, membros (atributos ou métodos) de um tipo, parâmetros ou variáveis locais.
 - programas são conjuntos de pacotes organizados
- **Nome simples e nome qualificado**
 - Nome simples é um identificador
 - Nome qualificado permite acesso aos membros de pacotes e aos membros de tipos de referência

Nomes e Pacotes

- **Pacote consiste de um número de unidades de compilação e tem um nome hierárquico**
 - O escopo dos nomes definidos no pacote é o próprio pacote.
 - Unidade de compilação é um código fonte em Java
 - **Um arquivo com código fonte deve possuir extensão .java**
 - deve conter no máximo uma classe pública cujo nome é o mesmo do arquivo sem a extensão
 - **No topo do arquivo fonte podem aparecer**
 - declaração de pacote com a palavra reservada package.
 - declaração de importação com a palavra reservada import.
 - definição de classe com a palavra reservada class.
 - **import permite que os tipos declarados em um pacote sejam conhecidos em outro pelo nome simples e não pelo qualificado.**

Classes

Classe Object

- Superclasse de todas as outras classes
 - uma variável do tipo Object pode manipular uma referência para qualquer objeto (classe ou array)
 - os tipos classe e array herdam os métodos de Object

Classe String

- instâncias dessa classe representam seqüências de caracteres Unicode.
 - um objeto String tem um **valor constante e imutável**
 - literais String são referências a instâncias dessa classe

Classes

Classe Abstrata

- Uma classe deve ser declarada abstrata, se ela não está completamente implementada:
 - ela contem métodos abstratos.
- Uma classe abstrata não pode ser instanciada:
 - pode ser estendida por uma subclasse que deve implementar todos os seus métodos abstratos (ou se tornar abstrata também).
- Classes abstratas são utilizadas para reunir classes em torno de métodos comuns.
 - por exemplo, métodos matemáticos!

Interface

Conceituação semelhante a da classe abstrata

- a interface não pode ter métodos concretos e só pode ter atributos constantes.

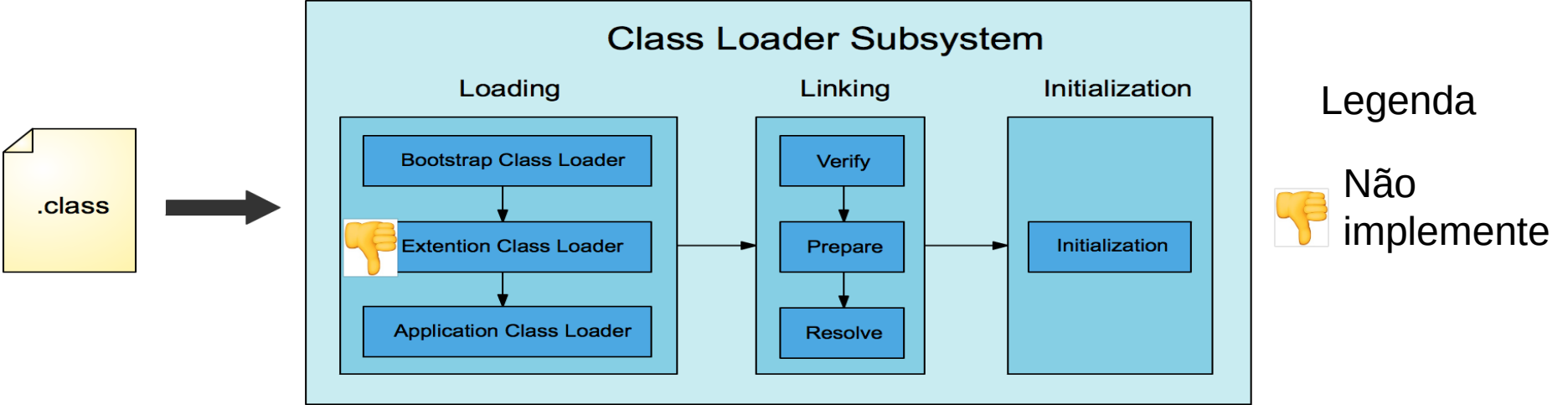
Pode ser usada para simular herança múltipla.

Representam idéias:

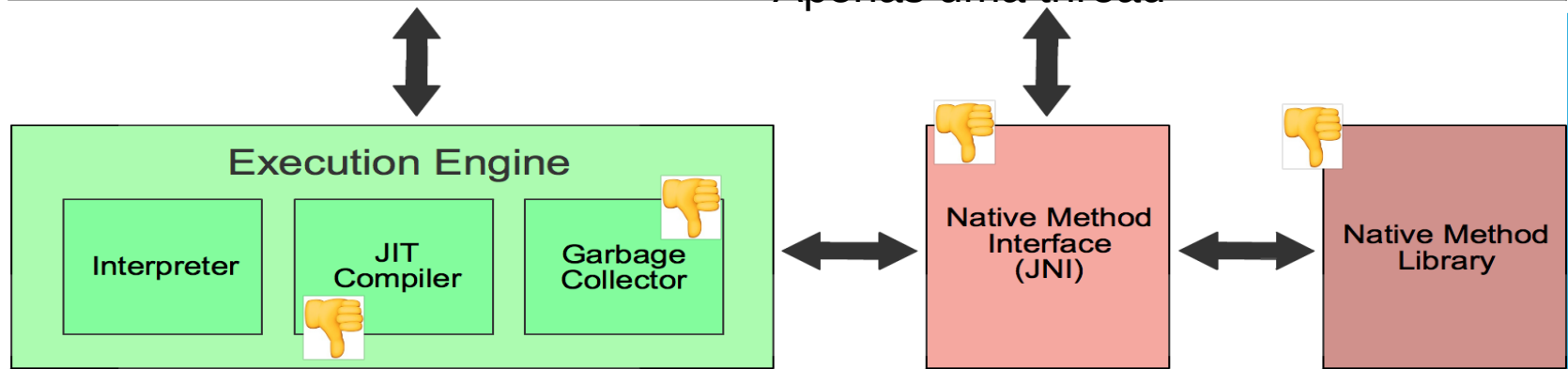
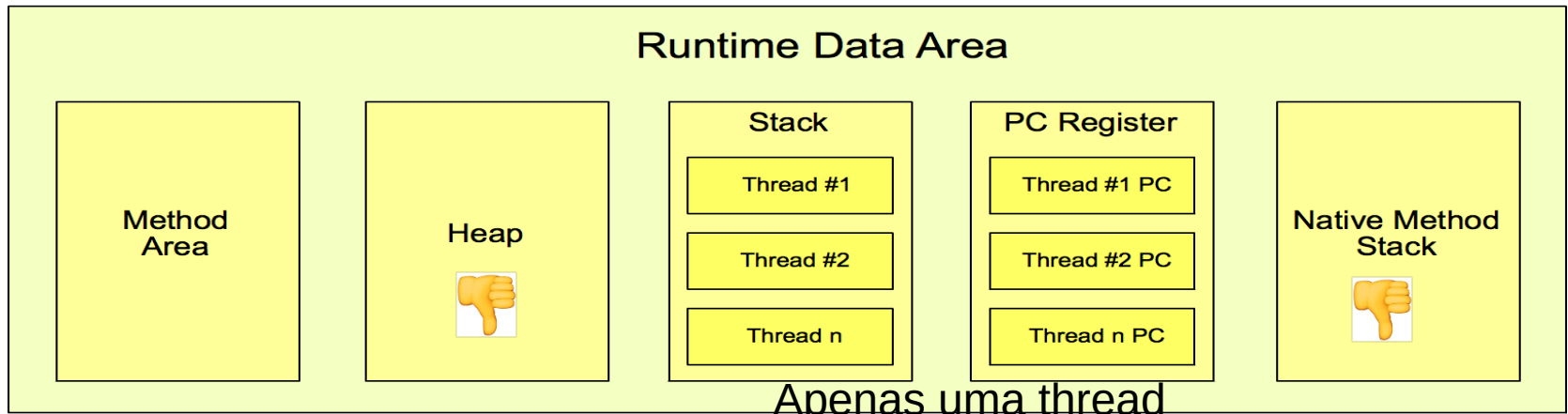
- não é considerada uma classe.
- as classes que implementam uma interface dão suporte a essa idéia.

Os métodos abstratos de uma interface podem ser utilizados se são definidos em uma classe que implementa a interface.

Todos os seus métodos e atributos são considerados públicos.



© Java Tutorial Network - <https://javatutorial.net>



Execução de um arquivo .class com a JVM java.exe

iniciação da máquina virtual

- Inicia com a chamada do método **main** de uma classe, passando um array de strings (**parâmetros da linha de comandos**)
 - *main* deve ser declarado **public static void**
- **Exemplo**
java Terminator Hasta la vista Baby!
 - Execução de **main** pela JVM **falha pois a classe Terminator não está carregada na memória**
 - **ClassLoader** é executado para carregar a classe Terminator
 - É carregada se não houver erro.

Iniciação da máquina virtual

Terminator deve ser ligada

- Verificação verifica se o código é bem formado, possui tabela de símbolos e segue os requisitos semânticos da JVM.
 - pode lançar um erro
- Na preparação o código estático da classe e as estruturas de dados internas da JVM, tal como as tabelas de métodos, são alocadas.
- A resolução de referências simbólicas é o processo de resolver as referências de **Terminator** para as outras classes e interfaces
 - as outras classes e interfaces mencionadas são carregadas e as referências são checadas.
 - Essa verificação é “estática” e feita o mais cedo possível.
 - Outra alternativa é realizá-la de **forma preguiçosa**, o mais tarde possível, somente quando as referências externas forem resolvidas.

Iniciação da máquina virtual

Terminator deve ser iniciado

- Consiste na execução dos procedimentos para **iniciar as variáveis de classe** e as **iniciações estáticas da classe Terminator**, **na ordem em que aparecem**.
 - Todas as **superclasses** de Terminator **devem ser iniciadas** antes, começando pelas diretas, recursivamente.
 - No caso mais simples, a única superclasse direta é **Object** que **deve ser iniciada** antes da classe Terminator o ser.
 - Esse processo recursivo **envolve carga, verificação e resolução de todas as superclasses** na hierarquia,
 - Erros podem ser lançados durante esse processo.

O método *main* de Terminator pode ser executado.

Possíveis erros na inicialização

Exceções que podem ser lançadas pelo **classloader**

- **ClassFormatError**
 - Código binário da classe ou interface compilada é malformatado.
- **UnsupportedClassVersionError**
 - O formato do código binário da classe ou interface não pode ser carregado porque usa uma versão antiga do formato .class.
- **ClassCircularityError**
 - A classe ou interface não pode ser carregada porque é uma das suas superclasses ou superinterfaces.
- **NoClassDefFoundError**
 - Não foi encontrada definição para uma classe ou interface requerida.

Possíveis erros na inicialização

Exceções que podem ser lançadas na ligação da classe

■ Verificação

- VerifyError
 - Código binário apresenta erro: opcode inválido, desvio para o meio de uma instrução, método com assinatura incorreta, ...

■ Resolução

- IllegalAccessException:
 - Referência a um símbolo não pode ser resolvida porque o atributo ou método foi declarado private, protected, ou acesso default (não público) ou a classe não é pública.
- InstantiationException:
 - Uma referência a uma classe ou interface abstrata foi encontrada em uma expressão de criação de instância de classe
- NoSuchFieldError:
 - Uma referência a um atributo inexistente de uma classe ou interface foi encontrada.
- NoSuchMethodError:
 - Uma referência a um método inexistente de uma classe ou interface foi encontrada